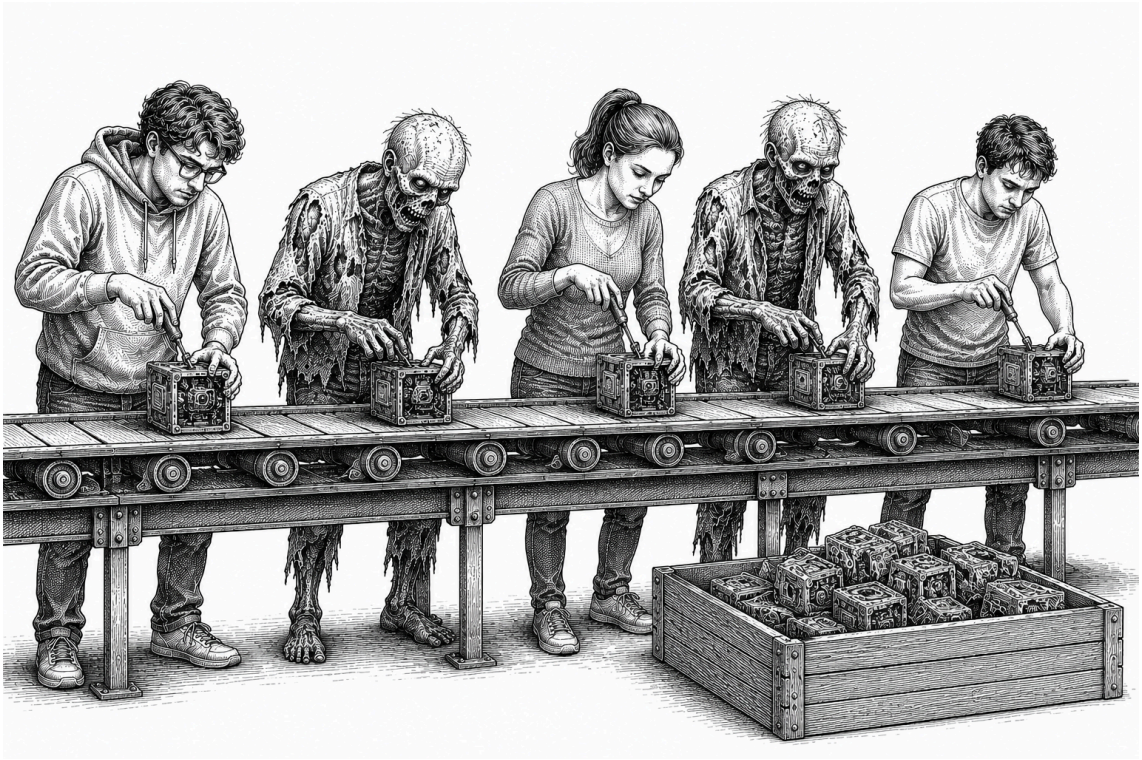
Data Parallelism with Rayon

Laboratory - Advanced Programming



HES-SO Valais//Wallis
Systems Engineering • Infotronics
Advanced Programming • Silvan Zahno
Fall Semester 2026 • I3

Silvan Zahno, **Rémy Borgeat**
07.08.2026 • v1.0 • final

Contents

1 Goal

Modern Central Processing Units (CPUs) have multiple cores, but most programs only use one. **Data parallelism** lets you harness all cores by processing chunks of data simultaneously.

In this laboratory you will refactor the Humans vs. Zombies simulation to use Rayon, Rust's data-parallelism library. You will learn to:

- Identify loops that can safely run in parallel
- Transform sequential iterators into parallel ones
- Recognise when a loop **cannot** be parallelised due to sequential dependencies
- Measure the speedup gained from parallel execution

1.1 What is Rayon?

Rayon turns sequential iterator chains into parallel ones. The Application Programming Interface (API) mirrors the standard iterator methods - instead of using **`iter()`**, you can use **`par_iter()`**.

```
// Sequential
let sum: u32 = numbers.iter().map(|n| n * n).sum();

// Parallel (just add par_)
let sum: u32 = numbers.par_iter().map(|n| n * n).sum();
```

Rayon automatically splits your data across available CPU threads, processes chunks in parallel, and collects the results transparently.

The key requirement is that the data type must be **`Send`** (safe to transfer between threads) and **`Sync`** (safe to share via references). You will add these bounds to the **`Entity`** trait.

Rayon is not a magic bullet. It only helps when the work per element is significant enough to amortise the overhead of parallelisation. Easy loops lose, heavy loops win - measure before parallelising.

1.2 Project Setup

The files for this lab are located in the **`aprog-labs`** repository in the **`labs/03-conc/02-hvz-rayon`** directory.

2 Refactor with Rayon

We will now refactor the Humans vs. Zombies simulation and implement parallelism with Rayon. The goal is to speed up the simulation by parallelising the counting, decay, and infection phases.

2.1 Baseline

First we need to establish a baseline for comparison. Run the benchmark on the current sequential version.

```
cargo run --release -- bench
```

- Run the benchmark on the sequential version.
  - Note the averages
- Fill out the Table 1 at the end of the document.

2.2 Refactoring

2.2.1 Add the Rayon Crate to the Project

To be able to use Rayon, we need to add it as a dependency in **`Cargo.toml`**. Run the following command:

```
cargo add rayon
```

The marker traits **`Send + Sync`** must be added to the **`Entity`** trait in **`entity.rs`**. It tells the compiler that every object that implements the **`Entity`** traits must also implement the **`Send`** and **`Sync`** traits. This allows the entities to be safely shared across threads.

Overview of Send and Sync Traits

`Send` and **`Sync`** are important *marker traits* (they do not have any methods or associated items) in Rust that help ensure thread safety in concurrent programming. They are automatically implemented for types that are safe to send or share between threads.

Definitions

- **`Send`**: A type is considered Send if it is safe to transfer ownership of that type to another thread. This means that the type can be moved across thread boundaries without causing data races or undefined behavior.
- **`Sync`**: A type is Sync if it is safe to share references of that type between threads. Specifically, if a type T is Sync, then a reference &T can be safely shared across threads.

More information under: Rust Book - Send and Sync Traits

- Add **rayon** to **Cargo.toml**.
- Add **Send + Sync** to the **Entity** trait in **entity.rs**.
- **cargo test** passes.

2.2.2 count_x() methods

In order to use Rayon in **world.rs** you need to import it:

```
1 use rayon::prelude::*;
```

Parallelise both methods **count_humans()** and **count_zombies()**

- Include rayon in **world.rs**.
- Parallelise both methods **count_humans()** and **count_zombies()**.
- **cargo test** passes.

2.2.3 tick_decay() method

Refactor the **for** loop to parallelise it.

- Parallelise **tick_decay()**.
- **cargo test** passes.

2.2.4 tick_infection()

Parallelise the check phase (read-only). The removal and creation loops must stay sequential.

- Parallelise the **tick_decay()** check phase.
- **cargo test** passes.

2.2.5 tick_movement() method

Can **tick_movement** be parallelised with Rayon? Why or why not? What would happen if two entities computed their positions simultaneously?

Think about the borrowing rules in Rust. A data can have either one mutable reference or any number of immutable references at a time.

- Understand why **`tick_movement`** stays sequential.
- All **`cargo test`** tests pass.

2.3 Benchmark

Run **`bench`** on your refactored version:

```
cargo run --release -- bench
```

Fill in your results and compare with your baseline:

Phase	Baseline (sequential)	Refactored (Rayon)	Δ [s] and [%]
movement			
infection			
decay			
Total			

Table 1 - Benchmark comparison (fill in your measurements)

Compare your measurements and answer:

1. Which phase benefits **most** from Rayon? Why?
2. Which phase gets **slower** with Rayon? Why?
3. Why doesn't movement change between the two versions?
4. What does this tell you about when parallelism is worth it?

3 Checkout

Before finishing, ensure you have completed:

Setup

- [ ] **`rayon`** added to **`Cargo.toml`**
- [ ] **`Send + Sync`** added to the **`Entity`** trait
- [ ] **`use rayon::prelude::*;`** in **`world.rs`**

Counting (**`count_humans`** / **`count_zombies`**)

- [ ] Parallelised with Rayon
- [ ] Tests pass

Decay (**`tick_decay`**)

- [ ] Parallelised with Rayon
- [ ] Tests pass

Infection (**`tick_infection`**)

- [ ] Check phase parallelised with Rayon
- [ ] Removal and creation loops stay sequential
- [ ] Tests pass

Movement (**`tick_movement`**)

- [ ] Understand why it stays sequential
- [ ] Tests pass

Benchmark

- [ ] Baseline recorded at start
- [ ] Refactored version benchmarked
- [ ] Table filled and analysis questions answered

Final verification

- [ ] **`cargo test`** reports all tests passing
- [ ] **`cargo run`** shows the game running correctly

4 Task Spawn (Optional)

This section is **optional** and lives in its **own** project. It does not depend on the Rayon refactoring - do it whenever you want a look under the hood.

Rayon made parallelism look effortless: swap **`iter()`** for **`par_iter()`** and you are done. But something has to create the threads, hand each one a slice of the work, and collect the results. In this bonus you build that machinery **by hand** with the standard library only - no Rayon, no external crates.

The task: count how many numbers in a large list are **prime**, spreading the work across several worker threads with **`std::thread::spawn`** and **`JoinHandle::join`**.

The prime numbers are in **`data/primes.txt`** (one per line). The sequential version is already implemented in **`src/counting.rs`** as **`count_primes_sequential()`**. Your job is to implement the parallel version **`count_primes_parallel()`**.

4.1 Project Setup

The files for this lab are in the **`aprog-labs`** repository under **`labs/03-conc/03-prime-tasks`**. It is a small, standalone project:

4.2 How `spawn` and `join` Work

`std::thread::spawn` starts a new thread that runs a closure, and immediately returns a **`JoinHandle<T>`**. Calling **`join()`** on that handle **blocks** until the thread finishes and gives you back the value the closure returned:

```rust
use std::thread;

let handle = thread::spawn(|| {
  // Everything inside happens on another thread
  40 + 2
});

// waits until tasks finishes, then returns 42
let result: i32 = handle.join().unwrap();
```

Two rules shape the whole exercise:

A spawned thread may outlive the function that created it, so its closure must be **`'static`** - it **cannot borrow** local data like a **`&[u64]`** slice. The simplest fix: give each thread its **own** copy of its chunk with **`chunk.to_vec()`** and **`move`** it into the closure.

4.3 Your Task: `count_primes_parallel()`

Open **`src/counting.rs`** and implement:

```
pub fn count_primes_parallel(numbers: &[u64], num_threads: usize) -> usize
```

The idea - **split, spawn, join, sum**:

1. Handle the edges: treat **`num_threads == 0`** as **`1`**, and return **`0`** for an empty slice (a zero-sized chunk would panic).
2. Pick a chunk size, e.g. **`numbers.len().div_ceil(num_threads)`**.
3. For each **`chunk`** of **`numbers.chunks(chunk_size)`**: copy it with **`chunk.to_vec()`**, **`move`** it into a **`thread::spawn`** closure that counts its primes with **`is_prime`**, and keep the returned handle in a **`Vec`**.
4. **`join()`** every handle and **sum** the partial counts.

The result must be identical to **`count_primes_sequential(numbers)`**.

- Implement **`count_primes_parallel()`** in **`src/counting.rs`** using only **`std::thread`**.
- Run **`just test`**: all 6 **`counting::`** tests (plus the **`primes::`** tests) pass.
- **`just run`** prints the number of primes found in the data file.

4.4 Benchmark

Compare the single-threaded baseline against your parallel version:

```
just bench      # or: cargo run --release -- bench
```

It prints both counts (they must match), each timing, and the measured speedup.

Run	Sequential [ms]	Parallel [ms]	Speedup
your machine			

Table 2 - Benchmark result (fill in your measurement)

Fill in Table 2 and answer:

1. Roughly how many worker threads did the program use, and why that number?
2. Is the speedup equal to the number of threads? What overheads keep it lower?
3. What would happen to the speedup if each number were tiny (say all **`< 100`**)?

Glossary

CPU **– Central Processing Unit**: The main processing component of a computer. 3, 3

API **– Application Programming Interface**: A set of rules and interfaces that allows software components to communicate. 3